GardenGroup Incident Management: Projectdocumentatie

Project NoSQL (1918IN233A), Deliverable 2 en 3
Repository: https://github.com/CarloLINH/GardenGroup

1. Introductie en sprintdoel

De Garden Group Incident Management applicatie ondersteunt de IT-servicedesk bij het registreren en verwerken van incidenten en het beheren van gebruikers. Het doel van deze deliverable 2 was het realiseren van de MongoDB CRUD-operaties (Create, Read, Update, Delete) voor zowel tickets als medewerkers, evenals het ontwikkelen van een dashboard dat inzicht biedt in de huidige status van tickets en de werkdruk. Dit document beschrijft de gezamenlijke architectuur, databasekeuzes en de specifieke implementaties van de Tickets-module, Employees-module en Dashboard-module.

Het doel van Deliverable 3 was het bouwen van de front-end en het koppelen van alle functionaliteit: inloggen met een rechtensysteem dat gewone medewerkers en Servicedesk-medewerkers onderscheidt, een gewone medewerker die tickets aanmaakt en zijn eigen tickets en dashboard ziet, een Servicedesk die tickets en medewerkers beheert en het dashboard van alle tickets ziet, en per student één eigen extra functionaliteit als aparte klasse. Deze onderdelen staan beschreven in sectie 8 t/m 18.

2. Database en technologiekeuze

De applicatie is gebouwd in C# (ASP.NET MVC) en maakt gebruik van een MongoDB database, verbonden via de officiële MongoDB.Driver NuGet-package. MongoDB is gekozen vanwege het flexibele documentmodel, wat ideaal is voor incidentbeheer en snelle data-opslag.

Documentstructuur en Relaties:

- **Tickets Collectie:** Tickets worden opgeslagen in een eigen collectie. Er is overwogen om tickets in te sluiten (embedding) in het Employee document, maar dit zou ertoe kunnen leiden dat de array ongecontroleerd groeit. Omdat de Service Desk tickets vaak onafhankelijk van de medewerker moet kunnen inzien, sluit een aparte collectie met referenties beter aan bij de toegang patronen.
- **Employees Collectie:** Bevat medewerkers gegevens (zoals naam, email, telefoonnummer en rol) in een flexibel JSON-formaat.
- **Dashboard Modellen:** Voor de dashboard statistieken worden de resultaten direct vanuit MongoDB gemapt naar C# typed model objects (Status Percentage and Employee Workload). Met [BetonElement("...")] attributen is de koppeling tussen afwijkende MongoDB-veldnamen en C#-properties expliciet gemaakt.
- **Connectie:** De connectionstring en database configuratie worden veilig opgehaald via een afzonderlijke Secret Service en .NET User Secrets. Hierdoor staan er geen wachtwoorden in de broncode of appsettings.json.

- **Indexing:** Alle indexen op tickets-en employees-collectie worden één keer bij het opstarten van de applicatie aangemaakt door Mongo Index Initializer.Ensure Indexes (aangeroepen in Program.cs).

3. CRUD-implementatie (Tickets & Employees)

Tickets Module

Deze module beheert de incidenten.

- **Create (Aanmaken):** Slaat een nieuw incident op in de database.
    - **Methode:** `TicketRepository.CreateTicket(Ticket ticket)`

```
public void CreateTicket(Ticket ticket)
{
    _ticketCollection.InsertOne(ticket);
}
```

Read (Ophalen): Haalt tickets uit de database voor de overzichten.

- **Methode:** `TicketRepository.GetAllTickets()` / `TicketRepository.GetTicketById(ObjectId id)`

```
public List<Ticket> GetAllTickets()
{
    return _ticketCollection.Find(t => true).ToList();
}

public Ticket GetTicketById(ObjectId id)
{
    return _ticketCollection.Find(t => t.Id == id).FirstOrDefault();
}
```

- **Update (Bewerken):** Wijzigt details of statussen van een bestaand ticket in de database.
    - **Methode:** `TicketRepository.UpdateTicket(Ticket ticket)`

```
public void UpdateTicket(Ticket ticket)
        {
            var filter = Builders<Ticket>.Filter.Eq(t => t.Id, ticket.Id);
```

```
            var update = Builders<Ticket>.Update
                .Set(t => t.Title, ticket.Title)
                .Set(t => t.Description, ticket.Description)
                .Set(t => t.AssignedToId, ticket.AssignedToId)
                .Set(t => t.Priority, ticket.Priority)
                .Set(t => t.Status, ticket.Status)
                .Set(t => t.UpdatedAt, DateTime.Now);

            if (ticket.Status == Statuses.Resolved && ticket.ResolvedAt == null)
            {
                update = update.Set(t => t.ResolvedAt, DateTime.Now);
            }
            if (ticket.Status == Statuses.Closed && ticket.ClosedAt == null)
            {
                update = update.Set(t => t.ClosedAt, DateTime.Now);
            }

            _tickets.UpdateOne(filter, update);
        }
```

- **Delete (Verwijderen):** Verwijder onjuiste of dubbele tickets uit de database.
    - **Methode:** `TicketRepository.DeleteTicket(string id)`

```
public void DeleteTicket(string id)
{
    var filter = Builders<Ticket>.Filter.Eq(t => t.Id, id);
    _ticketCollection.DeleteOne(filter);
}
```

Create (Aanmaken)

Voegt een nieuwe medewerker toe aan de database bij onboarding.

- **Klasse/Methode:** `EmployeeDao.Insert(Employee employee)` / `EmployeeService.CreateEmployee(...)`

```
public void CreateEmployee(Employee employee)
{
    _employeeCollection.InsertOne(employee);
}
```

Read (Ophalen)
Haalt alle medewerkers op voor het overzicht of zoekt een specifieke gebruiker op e-mailadres voor authenticatie.

- **Klasse/Methode:** `EmployeeService.GetAllEmployees()` / `EmployeeService.GetEmployeeByEmail(string email)`

```
public List<Employee> GetAllEmployees()
{
    return _employeeCollection.Find(e => true).ToList();
}

public Employee GetEmployeeByEmail(string email)
{
    return _employeeCollection.Find(e => e.Email == email).FirstOrDefault();
}
```

Update (Bewerken)
Wijzigt bestaande gegevens van een medewerker (zoals telefoonnummer, rol of locatie).

- **Klasse/Methode:** `EmployeeService.UpdateEmployee(Employee employee)`

```
public void UpdateEmployee(Employee employee)
{
    var filter = Builders<Employee>.Filter.Eq(e => e.Id, employee.Id);
    _employeeCollection.ReplaceOne(filter, employee);
}
```

Delete (Verwijderen)
Verwijder het profiel van een medewerker die de organisatie verlaat.

- **Klasse/Methode:** `EmployeeService.DeleteEmployee(ObjectId id)`

```
public void DeleteEmployee(ObjectId id)
{
    var filter = Builders<Employee>.Filter.Eq(e => e.Id, id);
    _employeeCollection.DeleteOne(filter);
}
```

4. Queries en aggregaties

Voor het zoeken en filteren van medewerkers (zoals zoeken op e-mail of ID) zijn directe MongoDB-filters en LINQ-expressies gebruikt. Queries bestaan voornamelijk uit enkelvoudige document-lookups en eenvoudige overzichtslijsten.

De lijst query voor tickets combineert filter, sortering en projectie in één `Find`. De volgende indexen ondersteunen die query's en worden aangemaakt in `Mongo Index Initializer:`

Index	Velden	Query waarvoor
`ix_status_createdAt`	status, createdAt	Tickets filteren op status, nieuwste eerst
`ix_assignedTo_createdAt`	assignedTo, createdAt	Tickets van één medewerker, nieuwste eerst
`createdAt_-1`	createdAt	Alle tickets op datum sorteren
`ix_reportedBy_status`	reportedBy, status	Dashboard: statuspercentages per melder
`ux_ticketId` (uniek)	ticketId	Unieke ticketnummers en snel het hoogste nummer ophalen

5. Alternatieven en afwegingen

Tickets Crud

Tijdens de ontwikkeling is er nagedacht over de manier waarop de applicatie communiceert met MongoDB. Hierbij is de afweging gemaakt tussen de huidige synchrone aanpak en een asynchroon alternatief.

Het Alternatief: Asynchrone methodes (`async` / `await`) In plaats van de huidige synchrone methodes zoals `ToList()`, `InsertOne()` en `ReplaceOne()`, biedt de MongoDB C# driver asynchrone alternatieven, zoals `ToListAsync()`, `InsertOneAsync()` en `ReplaceOneAsync()`. Dit betekent dat de database-aanroepen het sleutelwoord `await` krijgen en dat de functies een `Task` of `Task<T>` retourneren.

Voordelen van het asynchrone alternatief:

- **Betere schaalbaarheid:** Wanneer de applicatie een verzoek naar de database stuurt (een I/O-operatie), hoeft de server-thread niet te wachten tot het antwoord terug is. De thread komt vrij om andere web verzoeken af te handelen.

- **Performance:** Bij een applicatie met veel gelijktijdige gebruikers voorkomt dit een bottleneck en blijft de API veel responsiever.

Nadelen en de keuze voor de huidige (synchrone) opzet:

- **Complexiteit:** Asynchrone code werkt besmettelijk door de hele applicatie (ook wel "async all the way" genoemd). Als de *Repository* asynchroon wordt, moeten ook de *Services* en de *Controllers* asynchroon worden gemaakt.
- **Reden van huidige keuze:** Voor de huidige scope van dit project (waarbij de focus ligt op een werkende CRUD-architectuur en overzichtelijke code) is er doelbewust gekozen voor synchrone aanroepen. Dit houdt de code simpeler, beter leesbaar en makkelijker te debuggen.

Mocht de applicatie in de toekomst uitgroeien tot een groot platform met veel dataverkeer, dan is de overstap naar `async/await` het belangrijkste alternatief om door te voeren.

Employee Crud

- **Alternatief:** Alle medewerkers bij het opstarten in een C# `List<Employee>` laden en lokaal filteren met LINQ.
- **Gekozen oplossing:** Direct filteren en zoeken op de MongoDB-database met queryfilters (`Builders<Employee>.Filter`).
- **Afweging:** Server-side filteren voorkomt onnodig netwerkverkeer en hoog geheugengebruik in de applicatie zodra het aantal medewerkers groeit.

Tickets en indexen

Alternatief	Gekozen oplossing	Afweging
Losse enkelvoudige indexen op `status` en `assignedTo`	Samengestelde indexen met `createdAt` als tweede veld	Filteren en sorteren worden door één index afgehandeld. De enkelvoudige indexen waren overbodig en kosten alleen snelheid.
Indexen aanmaken in de `TicketRepository`-constructor	Eenmalig bij het opstarten in `Program.cs`	De repository is `Scoped` en wordt per request aangemaakt, dus de constructor zou bij elk request indexen aanmaken
Drie aparte methodes (op status, op medewerker, op datum)	Eén `GetTickets` met optionele filters	Filters zijn combineerbaar en er is geen dubbele code.
Alle tickets laden en het hoogste `TicketId` in C# bepalen	`GetLastTicketId` via de unieke index, met retry bij duplicate key	Constante kosten per insert en veilig bij gelijktijdig aanmaken, nadeel is een mogelijke herhaal poging.

Een aparte teller-collectie voor ticketnummers	Retry op de unieke index	Geen extra collectie nodig.Een teller is beter bij zeer veel gelijktijdige tickets.
Volledige tickets in de lijst ophalen	`description` weglaten met een projectie	Minder data per lijst verzoek. De beschrijving wordt pas op de detailpagina geladen
Tickets zonder behandelaar meetellen in de werkverdeling	`Niet meetellen`	Een werkverdeling per medewerker heeft geen betekenis voor onbehandelde tickets.
`$facet` of `$set Window Fields` (MongoDB 5.0+) in plaats van meerdere `$group`-stappen	`Losse $group-stappen`	De alternatieven kunnen korter zijn, maar zijn lastiger te lezen. We kozen voor duidelijke stappen die we in Compass één voor één konden controleren.

Gerelateerde medewerker ophalen

Beslissing: $lookup in MongoDB. Alternatief: het ticket ophalen en daarna reporter en assignee met twee extra Find-queries in C#. Afweging: drie queries zijn simpeler te lezen, de pipeline is één round trip maar vraagt handmatig mappen (GetFirstEmployee). Onderbouwing: door het verwijzing model is dit het enige scherm dat beide kanten toont, dus hier is de $lookup echt nodig. De lijstweergave gebruikt bewust geen $lookup.

6. Werkverdeling

Teamlid	Geïmplementeerde functionaliteit	Relevante bestanden / Klassen
Kyano	Complete Employees-module: datamodel, CRUD-operaties, DAO-/servicelaag en integratie met de gebruikersinterface.	Employee.cs, IEmployeeRepository.cs, IEmployeeService.cs, EmployeeRepository.cs, EmployeeService.cs

Frank	$lookup, errorhandling en integratie: bij ticket de reporter en assignee tonen, ongeldige ticket en employee-ids een nette error geven, ontbrekende tickets/medewerkers geven 404 en duplicaten voorkomen.	TicketRepository.GetTicketWithEmployees, TicketDetails, Views/Ticket/Details.cshtml ObjectId-controle in TicketRepository, EmployeeRepository, TicketController.Details Controllers, UpdateTicket, views MongoIndexInitializer, employeeservice,ticketservice.createticket
Carlo	Dashboard: aggregaties voor status, percentages en werkverdeling, controller en view.	`ITicketQueries.cs`, `TicketQueries.cs`, `StatusPercentage.cs`, `EmployeeWorkload.cs`, `DashboardViewModel.cs`, `DashboardController.cs`, `Views/Dashboard/Index.cshtml`
Mart	Indexing en `MongoIndexInitializer`. filteren en sorteren van tickets (status, medewerker, datum). Uniek ticketnummer met retry. Opschonen van `ITicketRepository`. README; reviewen en mergen van pull requests.	`MongoIndexInitializer.cs`, `Program.cs`, `TicketRepository`, `ITicketRepository`, `ITicketService`, `TicketService`,`TicketController.Index`, `Views/Ticket/Index.cshtml`, `README.md`

Emre	Basis Tickets-module: datamodel, CRUD-operaties, repository en integratie met de webinterface.	`TicketController.cs`, `TicketService.cs`, `TicketRepository.cs`, `ITicketService.cs`, `UI Views (Ticket: Create, Edit, Index, etc.)`

7. Testen en uitvoeren

Testen

De CRUD-functionaliteit en dashboard queries zijn gecontroleerd aan de hand van de MongoDB-database.

- **Tickets:** Het aanmaken, ophalen, bijwerken en verwijderen van tickets is gecontroleerd. Ook zijn filteren op status, filteren op toegewezen medewerker en sorteren op aanmaakdatum geïmplementeerd.
- **Medewerkers:** Het aanmaken, ophalen en bijwerken van medewerkers is geïmplementeerd. Daarnaast is gecontroleerd dat dubbele `employeeId`- en e-mail waarden worden voorkomen. Medewerkers kunnen worden gedeactiveerd zonder hun gegevens uit de database te verwijderen.
- **Ticket Details:** De reporter en assignee worden opgehaald met een MongoDB `$lookup` op de `employees`-collectie.
- **Dashboard:** De ticket status percentages worden berekend met een MongoDB-aggregatie pipeline. De werkverdeling per medewerker wordt berekend met aggregaties op basis van `$group` en `$lookup`.
- **Indexen:** De indexen worden automatisch aangemaakt wanneer de applicatie start.

Configuratie en uitvoeren

Vereisten

- .NET 10 SDK.
- Toegang tot een MongoDB-database via MongoDB Atlas of een lokale MongoDB-instantie.

1. Configureer de databaseverbinding

De connectionstring staat niet in de repository. Vraag een teamlid om de connectionstring en stel deze lokaal in via .NET User Secrets.

Open een terminal in de map waarin `GardenGroup.csproj` staat en voer het volgende commando uit:

```
dotnet user-secrets set "ConnectionStrings:MongoDb"
"<your-mongodb-connection-string>"
```

Vervang `<your-mongodb-connection-string>` door de connectionstring van de database. Commit deze waarde nooit naar GitHub.

2. Start de applicatie

Voer vanuit de map met `GardenGroup.csproj` het volgende commando uit:

```
dotnet run
```

3. Database en indexen

De applicatie gebruikt de MongoDB-database `helpdesk` met de collecties `tickets` en `employees`. De benodigde indexen worden automatisch aangemaakt bij het opstarten van de applicatie.

8. Wijzigingen ten opzichte van Deliverable 2

De tekst van sectie 2 t/m 7 is ongewijzigd. De code is sinds Deliverable 2 op deze punten veranderd:

- **UpdateTicket (sectie 3):** de huidige methode wijzigt ook `Type` en `Deadline`. De tijdstempels `ResolvedAt`, `ClosedAt` en nieuw `EscalatedAt` worden alleen gezet als de status daadwerkelijk verandert (de repository leest eerst het huidige ticket). Daarnaast bestaat nu `UpdateStatus(id, expectedCurrent, newStatus)`, dat een status alleen wijzigt als het ticket nog de verwachte status heeft.
- **CreateTicket (sectie 3):** de service (`TicketService.CreateTicket`) zet `CreatedAt` en het ticketnummer (hoogste `ticketId` + 1, tot 3 pogingen bij een duplicate-key-fout). De repository controleert daarna dat `TicketId` en `ReportById` groter dan 0 zijn en voegt het ticket toe.
- **DeleteTicket (sectie 3):** de repository controleert eerst of het ID een geldige ObjectID is en verwijdert daarna met `DeleteOne`. In de applicatie gebeurt dit pas na een bevestigingspagina.
- **Employees (sectie 3):** de medewerkers-CRUD staat in `EmployeeRepository` en `EmployeeService` en is asynchroon (`InsertAsync`, `GetByEmailAsync`, `GetAllAsync`, `ReplaceAsync`). De klasse `EmployeeDao` bestaat niet meer. “Verwijderen” is nu deactiveren (`DeactivateAsync`, zie sectie 12). De service voegt duplicaat controle en wachtwoord hashing toe.
- **Indexen (sectie 4):** er zijn indexen bij gekomen, zie sectie 14.
- **Dashboard:** de categorieën en de regels erachter zijn vastgelegd, zie sectie 13.

9. Architectuur

De applicatie gebruikt gelaagde MVC: View < Controller > Service > Repository > MongoDB.

- Controllers verwerken HTTP-verzoeken, lezen de ingelogde gebruiker en kiezen de view.
- Services bevatten de bedrijfsregels (bijvoorbeeld: een ticket kan alleen worden toegewezen aan een actieve medewerker).
- Repositories bevatten alle MongoDB-queries.

Regels in services blijven buiten de controllers en zijn los van een web verzoek te testen. De lagen konden ook parallel door verschillende teamleden worden gebouwd. Een alternatief zijn queries direct in de controllers. Dat is sneller te schrijven, maar regels worden dubbel gebouwd en een regel als “alleen eigen tickets” is dan makkelijker op één plek te vergeten.

De keuze voor synchrone ticket methodes uit sectie 5 blijft staan. Medewerkers, inloggen, zoeken, sorteren op prioriteit en het dashboard is wel asynchroon. De combinatie is een bekende inconsistentie: ticket-CRUD async maken betekent dat repository, service en controller tegelijk moeten veranderen.

10. Authenticatie en rechtenbeheer

Uitbreiding van het datamodel. `Employee` heeft er de velden `passwordHash`, `isActive`, `passwordResetLink` en `passwordResetTime` bij. `Ticket` heeft er `type`, `deadline`, `priority` (getal: Low 0, Medium 1, High 2, Urgent 3) en `escalatedAt` bij. Prioriteit staat als getal opgeslagen, zodat sorteren in de database klopt (sorteren op tekst zou alfabetisch gaan).

Inloggen. `AccountController.Login` controleert e-mail en wachtwoord via `EmployeeService.AuthenticateAsync`. Onbekende gebruikers, inactieve gebruikers en foute wachtwoorden krijgen dezelfde algemene foutmelding, zodat niemand kan achterhalen welke e-mailadressen bestaan. Na een geslaagde login wordt een cookie gezet met vier claims: medewerkernummer, naam, e-mail en rol. De loginpagina accepteert alleen een lokale return-URL (`Url.IsLocalUrl`) en elke POST heeft een anti-forgery-token.

Wachtwoorden hashen. Wachtwoorden worden gehasht met `PasswordHasher<Employee>`. De migratie `EmployeePasswordMigration` draait bij het opstarten: ze zet een oud platte-tekstveld `password` om naar `passwordHash` en verwijdert daarna het oude veld. Alternatieven: een eigen SHA-hash (onveilig, geen salt) of volledige ASP.NET Identity (vraagt een gebruiker opslag en veel extra structuur voor dit project).

Rollen. `AppRoles` definieert `employee`, `agent` en `manager`. Agent en manager zijn Service Desk. De extensie-metoden `User.Is ServiceDesk()` en `User.GetEmployeeId()` lezen de claims. De rechten worden op drie lagen afgedwongen:

1. `[Authorize]` op elke controller, plus `ServiceDesk` op de Servicedesk-acties (manage, edit, delete, transfer, escalate, close). Wie geen rechten heeft, ziet de pagina “Acces denied”.
2. De gebruikers-ID komt altijd uit de claims, nooit uit een URL of formulierveld. De ticket lijst, zoekresultaten en het dashboard van een gewone medewerker worden op het eigen ID gefilterd.
3. De detailpagina controleert authority dus het openen van andermans ticket via de URL geeft 403.

Het navigatiemenu in `_Layout.cs.html` is rol afhankelijk (“My tickets” tegenover “Tickets”; “Employees” alleen voor Service Desk). Alternatief: `[Authorize(Roles = agent,manager)]` bij elke actie. We kozen voor één benoemd policy, zodat de definitie van “Service Desk” op één plek staat.

11. Tickets in de applicatie

Flow gewone medewerker. Inloggen > “My tickets” > “New ticket” > aanmaken > ticket in de lijst zien > details openen. De melder wordt automatisch ingevuld met de ingelogde gebruiker en is niet aan te passen in het formulier.

Flow Service Desk. Alle tickets (met filters en zoeken) > ticket aanmaken namens een medewerker (medewerker keuzelijst) > details openen > edit, transfer, escalate, close of delete na een bevestigingspagina.

Keuzes:

- Validatie staat in data annotations op het `Ticket`-model met duidelijke meldingen (verplichte velden, minimale en maximale lengte). De deadlineregels (verplicht, niet in het verleden) worden in de controller gecontroleerd. De server is source of truth, want validatie in de browser is te omzeilen.
- Over-posting-bescherming: de POST-acties gebruiken `[Bind]` met een expliciete lijst velden, zodat een gebruiker geen velden meestuurt die hij niet mag wijzigen.
- Ongeldig of ontbrekend ID: een ongeldig ID geeft 400 en een ontbrekend ticket geeft 404.
- Toewijzing: in het bewerkscherm kan alleen aan een actieve Service Desk-medewerker worden toegewezen; bij een ticket namens iemand moet de melder een actieve medewerker zijn.

12. Medewerkersbeheer en besluit over deactiveren

Service Desk kan medewerkers tonen, aanmaken, bewerken en deactiveren.

- Duplicaatbeveiliging op drie plekken: de controller (nette veldmeldingen), de service (voorkomt omzeilen van de UI) en unieke indexen in MongoDB (de enige bescherming bij twee gelijktijdige verzoeken).
- Medewerkernummer-suggestie: het formulier stelt het hoogste nummer + 1 voor.
- Wachtwoord: verplicht bij aanmaken, optioneel bij bewerken (leeg = ongewijzigd). Een nieuw wachtwoord wist ook een openstaand resettoken.

Wat gebeurt er met de tickets van een gedeactiveerde medewerker?

“Medewerker verwijderen” is geïmplementeerd als deactiveren (soft delete): `isActive` wordt `false` en het record blijft bestaan. Reden: tickets verwijzen naar medewerkers met `employeeId`, dus een harde delete zou tickets achterlaten met een melder of behandelaar die niet meer bestaat. Concreet:

- De medewerker kan niet meer inloggen en zijn wachtwoord niet meer resetten.
- Zijn tickets blijven bestaan en worden **niet** automatisch opnieuw toegewezen. Zijn naam blijft zichtbaar op ticketdetails en in de werkverdeling van het dashboard (de `$lookup` vindt het record nog).
- Hij verschijnt niet meer in de keuzelijsten voor melder, behandelaar of overdracht, en de servicelaag weigert een overdracht aan een inactieve of onbekende medewerker.
- Service Desk kan de medewerker heractiveren via het bewerkscherm.

Alternatieven: harde delete (breekt verwijzingen) of alle open tickets bij deactiveren opnieuw toewijzen (vraagt een regel voor wie ze krijgt).

13. Statusdefinitie en dashboard

De enum `Statuses` (Open, In_Progress, Escalated, Resolved, Closed) is de enige plek waar statusnamen bestaan. Repository, escalatieregels, views en de dashboardpipeline gebruiken die enum. Het dashboard groepeert ze in drie categorieën (`TicketCategory`):

Dashboardcategorie	Regel
Open (nog niet opgelost)	Status Open, In progress of Escalated (ook de standaard voor alles wat onbekend is)
Opgelost (gesloten en opgelost)	Status Resolved, of `resolvedAt` is gevuld, zodat een ticket dat eerst opgelost en daarna gesloten is, als opgelost blijft tellen
Gesloten zonder oplossing	Status Closed en geen `resolvedAt`

Een ticket met status Open telt altijd als Open, ook als er nog een oude `resolvedAt` staat (heropend ticket).

Statuspercentages

(`TicketQueries.GetStatusPercentagesAsync`): `$match` op melder (alleen voor gewone medewerkers, zodat `ix_reportedBy_status` bruikbaar is) > `$group` per categorie met `$switch` > `$group` voor het totaal > `$unwind` > `$project` met `$round(aantal / totaal × 100, 1)`. De berekening gebeurt in de database, dus er

komen maar een paar rijen bij de applicatie aan. Categorieën zonder tickets worden met 0 toegevoegd, en bij nul tickets toont de pagina een lege-toestand-melding, zonder deling door nul of een kapotte lege grafiek.

Werkverdeling

(`GetEmployeeWorkloadsAsync`): `$match` op tickets met een behandelaar > `$group` per `assignedTo` > `$lookup` op `employees` > `$project` van de naam (of “Unknown”) > `$sort` aflopend. Eerst groeperen en dan pas `$lookup` betekent één lookup per medewerker in plaats van één per ticket.

Wie ziet wat

Een gewone medewerker ziet alleen de eigen percentages, met het ID uit de claims. Service Desk ziet alle tickets en de werkverdeling; een gewone medewerker krijgt de werkverdelingsdata nooit.

14. Aanvullende queries en indexen

`TicketFilterBuilder` bouwt het filter op status / behandelaar / melder op één plek, voor de ticketlijst, de prioriteitssortering en het zoeken. Zo kunnen die drie niet verschillend filteren en wordt de beperking voor gewone medewerkers niet vergeten.

De standaardlijst is niet gepagineerd. Zoeken en sorteren op prioriteit geven pagina’s van 100 (paginanummer en paginagrootte worden gevalideerd), met een vaste sorteervolgorde zodat een ticket niet dubbel verschijnt of verdwijnt tussen pagina’s.

Bij het opstarten worden de oude enkelvoudige indexen `status_1` en `assignedTo_1` verwijderd, omdat de samengestelde indexen ze afdekken. De indexen uit sectie 4 zijn aangevuld met:

Index	Velden	Query waarvoor
ix_reportedBy_createdAt	reportedBy, createdAt	“My tickets” voor een gewone medewerker
ix_priority_sort	priority , createdAt , _id	Sorteren op prioriteit (beide richtingen)
ix_reporter_priority_sort	reportedBy, priority , createdAt , _id	Prioriteitssortering van eigen tickets
ix_status_priority_sort	status, priority , createdAt , _id	Prioriteitssortering binnen een status
ux_employeeId (uniek)	employeeId	Duplicaatbeveiliging
ux_employee_email (uniek)	email	Duplicaatbeveiliging

15. Individuele functionaliteit en werkverdeling Deliverable 3

Elke functionaliteit bestaat als aparte klasse met een interface en is geregistreerd in `Program.cs`.

Wachtwoord vergeten: Kyano.

Classes: `ForgotPasswordService` / `IForgotPasswordService`; `AccountController.ForgotPassword` en `ResetPassword`; drie views (MVC-patroon). De service maakt een willekeurig token van 32 bytes (`RandomNumberGenerator`) met een geldigheid van 30 minuten. Bij het resetten worden tokens in constante tijd vergeleken (`CryptographicOperations.FixedTimeEquals`), het nieuwe wachtwoord wordt als hash opgeslagen en het token wordt gewist, dus een link werkt maar één keer. De pagina geeft hetzelfde antwoord, of het e-mailadres nu bestaat of niet. *Huidige beperking:* de applicatie maakt de resetlink aan maar verstuurt geen e-mail; de link staat op de bevestigingspagina, bedoeld voor een latere mailkoppeling. *Alternatieven:* de token provider van ASP.NET Identity (vraagt een Identity-opslag); een hash van het token opslaan in plaats van het token zelf (veiliger bij een databaselek; wij sloegen het token op om de flow simpel te houden).

Sorteren op prioriteit: Carlo.

Classes: `TicketPrioritySorter` / `ITicketPrioritySorter`, `TicketSort`, `TicketSortOrder`. De sortering draait in MongoDB en combineert met alle filters. `createdAt` en `_id` zijn secundaire sleutels, zodat gelijke prioriteiten een vaste volgorde hebben bij het pagineren. Laag-naar-hoog is de exacte omgekeerde van hoog-naar-laag, dus dezelfde index bedient beide richtingen. Alternatief: alles laden en sorteren in C# (schaalt niet en breekt het pagineren).

Escaleren / sluiten: Emre.

Classes: `TicketEscalator` / `ITicketEscalator`; acties `Escalate` en `Close`. De regels staan in twee kleine methodes, `CanEscalate` (Open of In progress) en `CanClose` (alles behalve Closed). Escaleren betekent “naar het management sturen”: status Escalated plus `escalatedAt`. De repositoryupdate `UpdateStatus` wijzigt alleen een ticket dat nog de status heeft die we lazen (`status == expectedCurrent`), zodat twee Service Desk-medewerkers die tegelijk handelen elkaar niet overschrijven. Het resultaat is een enum (`Done`, `NotFound`, `NotAllowed`) die de controller omzet in een melding. Beide acties zijn POST met anti-forgery-token en een bevestigingsdialoog.

Zoeken: Mart.

Classes: `TicketSearcher` / `ITicketSearcher`, `TicketListQuery`. Zoeken werkt op titel en beschrijving, ondersteunt AND en OR en sorteert met de nieuwste bovenaan. `printer AND offline OR wifi` wordt (printer AND offline) OR (wifi). Elk woord wordt met `Regex.Escape` onschadelijk gemaakt, zodat gebruikers geen reguliere expressie kunnen injecteren. De invoer is beperkt tot 100 tekens en 20 woorden. Het rolfilter (eigen

tickets voor een gewone medewerker) wordt met AND aan het zoekfilter gekoppeld, dus zoeken kan nooit tickets tonen die de gebruiker niet mag zien. `ParseSearchText` is publiek en statisch, zodat het zonder database te testen is. *Alternatief:* een MongoDB-tekstindex met `$text`; die ondersteunt AND/OR minder direct en zoekt alleen op hele woorden. Nadeel van onze aanpak: regexfilters kunnen geen index gebruiken, wat bij dit datavolume acceptabel is.

Ticket overdragen: Frank.

Classes: `TicketTransferService` / `ITicketTransferService`, `TicketRepository.TransferTicket`, `TicketController.Transfer`, `Views/Ticket/Transfer.cshtml`. De servicelaag (niet alleen de UI) weigert een doelmedewerker die niet bestaat of inactief is. De repository geeft het `UpdateResult` van MongoDB terug: het filter sluit tickets uit die al aan die medewerker zijn toegewezen, dus `MatchedCount == 0` betekent “niets te wijzigen” of “ticket niet gevonden”, en de service controleert welke van de twee. De controller toont voor elk geval een eigen melding. *Alternatief:* altijd de update uitvoeren zonder het `Ne`-filter (simpeler, maar dan is een overdracht aan dezelfde persoon niet te onderscheiden van een echte wijziging).

Aanvulling op de werkverdeling (sectie 6) voor Deliverable 3:

Teamlid	Extra werk in Deliverable 3	Individuele functionaliteit
Kyano	Login, rollen in de claims, wachtwoordhash en migratie, medewerkersbeheer-schermen, deactiveren. Bestanden: `AccountController.cs`, `EmployeesController.cs`, `Views/Account`, `Views/Employees`, `EmployeePasswordMigration.cs`	Wachtwoord vergeten (`ForgotPasswordService`)
Frank	Validatie van toewijzing aan actieve medewerkers, foutafhandeling bij ongeldige en ontbrekende ID’s. Bestanden: `TicketTransferService.cs`, `TicketController.Transfer`, `Views/Ticket/Transfer.cshtml`	Ticket overdragen (`TicketTransferService`)
Carlo	Persoonlijk dashboard voor gewone medewerkers (ID uit de claims), nul-ticketsafhandeling, prioriteitssortering. Bestanden: `TicketPrioritySorter.cs`, `ITicketPrioritySorter.cs`, `Views/Dashboard/Index.cshtml`	Sorteren op prioriteit (`TicketPrioritySorter`)

Mart	Rolhulpclasses, “My tickets”, filters voor beide rollen, indexen, README. Bestanden: `AppRoles.cs`, `UserClaimsExtensions.cs`, `TicketListQuery.cs`, `TicketFilterBuilder.cs`, `TicketSearcher.cs`, `ITicketSearcher.cs`	Zoeken met AND/OR (`TicketSearcher`)
Emre	Ticketformulieren met validatie, ticket namens een medewerker, bewerken met statuswijziging, verwijderen met bevestiging, gedeelde layout en styling. Bestanden: `Ticket.cs`, `TicketController.cs`, `Views/Ticket`, `_Layout.cshtml`, `site.css`, `TicketEscalator.cs`, `ITicketEscalator.cs`, `TicketUIExtension.cs`	Escaleren / sluiten (`TicketEscalator`)

Toelichting per teamlid op eigen keuzes.

Kyano: ontwerpkeuzes en onderbouwing

1. Wachtwoorden hashen met PasswordHasher in plaats van zelf een hash maken

Keuze: Voor het opslaan van wachtwoorden is `PasswordHasher<Employee>` uit ASP.NET Core gebruikt.

Wat: Wanneer een medewerker wordt aangemaakt of zijn wachtwoord wijzigt, wordt het wachtwoord gehasht voordat het in MongoDB wordt opgeslagen. Bij het inloggen wordt het ingevoerde wachtwoord gecontroleerd met de opgeslagen hash.

Waarom: Wachtwoorden mogen nooit als gewone tekst in de database worden opgeslagen. Door gebruik te maken van de bestaande PasswordHasher worden wachtwoorden op een veilige manier verwerkt, zonder zelf een hashingmechanisme te hoeven ontwikkelen.

Alternatief: Zelf een SHA256-hash implementeren. Dit is eenvoudiger, maar minder veilig omdat een gewone SHA256-hash geen ingebouwde salt en geschikte wachtwoordverwerking biedt. Een ander alternatief is volledige ASP.NET Identity, maar dat brengt voor dit project veel extra configuratie en databasecomplexiteit met zich mee. Daarom is gekozen voor PasswordHasher.

2. Soft delete gebruiken bij het verwijderen van medewerkers

Keuze: Medewerkers worden niet daadwerkelijk uit MongoDB verwijderd, maar gedeactiveerd met `isActive = false`.

Wat: Wanneer een Service Desk-medewerker een medewerker verwijdert, blijft het document in de employees-collectie bestaan. De medewerker kan niet meer inloggen en verschijnt niet meer in keuzelijsten. Service Desk kan de medewerker later opnieuw activeren.

Waarom: Tickets bevatten verwijzingen naar medewerkers via hun employeeId. Bij een harde verwijdering zouden bestaande tickets kunnen verwijzen naar een medewerker die niet meer bestaat. Door de medewerker te deactiveren blijven historische ticketgegevens intact en blijven namen zichtbaar bij ticketdetails en dashboardgegevens.

Alternatief: De medewerker volledig verwijderen uit MongoDB. Dit bespaart opslagruimte, maar kan problemen veroorzaken bij bestaande tickets en verwijzingen. Daarom is gekozen voor soft delete.

3. Duplicaatcontrole op meerdere lagen

Keuze: Het voorkomen van dubbele e-mailadressen en employeeId's gebeurt op drie verschillende niveaus: controller, service en MongoDB met unieke indexen.

Wat: De controller controleert de invoer en toont duidelijke foutmeldingen bij dubbele gegevens. De service voert dezelfde controle uit, zodat de bedrijfsregel niet afhankelijk is van de gebruikersinterface. MongoDB heeft daarnaast unieke indexen op employeeId en email.

Waarom: Alleen controleren in de controller is niet voldoende. Een service kan ook vanuit een andere plek worden aangeroepen. Bovendien kunnen twee gebruikers vrijwel tegelijkertijd hetzelfde e-mailadres proberen te registreren. De unieke index in MongoDB voorkomt dat beide records alsnog worden opgeslagen.

Alternatief: Alleen controleren in de controller of service. Dit is eenvoudiger, maar niet volledig betrouwbaar bij gelijktijdige verzoeken. Daarom is gekozen voor controle op alle drie de niveaus.

4. Algemene foutmelding bij het inloggen

Keuze: Bij een mislukte login wordt altijd dezelfde algemene foutmelding getoond.

Wat: Of een gebruiker niet bestaat, gedeactiveerd is of een verkeerd wachtwoord invoert, de applicatie toont dezelfde melding.

Waarom: Hiermee voorkomen we dat iemand kan achterhalen welke e-mailadressen daadwerkelijk geregistreerd zijn in de applicatie. Dit verkleint de kans op misbruik waarbij aanvallers geldige accounts proberen te verzamelen.

Alternatief: Voor iedere situatie een aparte foutmelding tonen, zoals 'e-mailadres bestaat niet' of 'wachtwoord incorrect'. Dit is gebruiksvriendelijker, maar geeft ook informatie aan mogelijke aanvallers. Daarom is gekozen voor één algemene foutmelding.

5. Rollenbeheer met claims en policies

Keuze: Voor het onderscheiden van gewone medewerkers en Service Desk-medewerkers gebruiken we claims in de authenticatiecookie en een centrale Service Desk-policy.

Wat: Na een succesvolle login worden onder andere het medewerkernummer, de naam, het e-mailadres en de rol opgeslagen als claims. Met de rollen `employee`, `agent` en `manager` kan de applicatie bepalen welke onderdelen een gebruiker mag bekijken en gebruiken.

Waarom: Hierdoor hoeft de applicatie niet bij iedere pagina opnieuw de medewerker op te zoeken om zijn rechten te controleren. Daarnaast staat de definitie van Service Desk op één centrale plek. Zowel agent als manager krijgt hiermee toegang tot de Service Desk-functionaliteiten.

Alternatief: Bij iedere controlleractie handmatig controleren welke rol de gebruiker heeft. Dit zorgt voor veel herhaalde code en vergroot de kans dat ergens een controle wordt vergeten. Daarom is gekozen voor centrale policies en autorisatie-attributen.

6. Wachtwoord vergeten met een tijdelijk en eenmalig token

Keuze: Voor het resetten van wachtwoorden is een eigen `ForgotPasswordService` gemaakt met cryptografisch willekeurige tokens.

Wat: De service genereert een token van 32 bytes met `RandomNumberGenerator`. Het token is 30 minuten geldig. Bij het resetten wordt het token gecontroleerd met `CryptographicOperations.FixedTimeEquals`. Na een succesvolle reset wordt het nieuwe wachtwoord gehasht opgeslagen en het reset-token verwijderd.

Waarom: Een wachtwoordreset moet veilig zijn en mag niet onbeperkt gebruikt kunnen worden. Door een willekeurig token met een beperkte geldigheid te gebruiken, verkleinen we de kans dat iemand ongeautoriseerd een wachtwoord kan wijzigen. Het token kan bovendien maar één keer gebruikt worden.

Alternatief: De ingebouwde tokenfunctionaliteit van ASP.NET Identity gebruiken. Dit is een goede oplossing, maar vereist voor dit project een uitgebreidere Identity-structuur. Daarom is gekozen voor een aparte service. Een toekomstige verbetering zou zijn om niet het volledige token, maar alleen een hash ervan in de database op te slaan.

7. Asynchrone databaseoperaties bij medewerkersbeheer

Keuze: Voor de Employee-module zijn asynchrone MongoDB-methodes gebruikt, zoals `InsertAsync`, `GetByEmailAsync`, `GetAllAsync` en `ReplaceAsync`.

Wat: Databasebewerkingen worden uitgevoerd met asynchrone methodes, zodat de applicatie niet onnodig hoeft te wachten tijdens databaseverzoeken.

Waarom: Wanneer de applicatie meerdere gebruikers tegelijkertijd verwerkt, hoeft een serverthread niet geblokkeerd te blijven tijdens het wachten op een databaseantwoord. Dit helpt bij de schaalbaarheid en responsiviteit van de applicatie.

Alternatief: Synchrone databaseaanroepen gebruiken, zoals bij de oorspronkelijke Tickets-module. Dit is eenvoudiger te implementeren, maar kan bij veel gelijktijdige verzoeken minder efficiënt zijn. Daarom is voor de Employee-module gekozen voor async/await.

Frank: ontwerpkeuzes en onderbouwing

1. Eén aggregatie met twee `$lookup`-stappen voor reporter en assignee

- *Keuze:* één pipeline die het ticket ophaalt en beide medewerkers koppelt.
- *Alternatieven:* twee extra `Find`-aanroepen, of de namen in het ticket inbedden.
- *Onderbouwing:* één databasecall is sneller dan drie roundtrips, en de joins gebeuren in MongoDB. Inbedden is minder geschikt, omdat de namen verouderen als een medewerker van naam verandert of inactief wordt. Dan moet je alle tickets van die medewerker bijwerken. Met `$lookup` blijft de medewerkers collectie de enige bron van waarheid.

2. Filter en update in één `UpdateOne`

- *Keuze:* het filter is `Id == ticketId AND AssignedToId != employeeId`, en de update gebeurt in dezelfde bewerking.
- *Alternatief:* eerst het ticket lezen, in code controleren en daarna opslaan.
- *Onderbouwing:* controle en wijziging zijn dan één atomaire bewerking. Er zit geen leesmoment tussen waarop een andere gebruiker het ticket kan wijzigen, en je bespaart een extra query.

3. `UpdateResult` teruggeven in plaats van een `bool`

- *Keuze:* de repository geeft `MatchedCount` en `ModifiedCount` door aan de service.
- *Onderbouwing:* “geen match” kan betekenen dat het ticket niet bestaat of dat het al aan deze medewerker was toegewezen. Met `UpdateResult` kan de aanroeper die gevallen apart behandelen en de gebruiker een passende melding geven. Een `bool` zou ze samenvoegen.

4. Validatie in de service, niet alleen in de UI

- *Keuze:* de service controleert of de medewerker bestaat en actief is.
- *Onderbouwing:* de dropdown toont alleen actieve medewerkers, maar een POST kan ook buiten het formulier om worden verstuurd, met een andere `employeeId`. De UI-controle is dus gemak voor de gebruiker, en de servicecontrole is de echte bescherming. Zo geldt de regel ook als de transfer later vanuit een andere plek wordt aangeroepen.

5. Foutafhandeling in lagen

- *Keuze:* de repository valideert de invoer, de service gooit betekenisvolle exceptions (`KeyNotFoundException`, `InvalidOperationException`), en de controller vertaalt die naar `BadRequest`, `NotFound` of een melding in het formulier.
- *Onderbouwing:* elke laag doet alleen wat bij zijn verantwoordelijkheid past. De service kent geen HTTP-statuscodes, en de controller kent geen databaseregels.

6. Ontbrekende gegevens afvangen

- *Keuze:* een lege `$lookup`-array wordt `null`, en de view toont “Unassigned” of “Unknown employee”.
- *Onderbouwing:* een ticket zonder behandelaar, of met een verwijderde medewerker, mag de detailpagina niet laten crashen.

Carlo: ontwerpkeuzes en onderbouwing

1. De berekening gebeurt in MongoDB
Wat: `GetStatusPercentagesAsync` voert één aggregatiepipeline uit die per ticket de categorie bepaalt (Open, Resolved, Closed zonder oplossing), telt en het percentage uitrekent. Er komen maximaal drie rijen naar de applicatie.
Waarom: Het aantal tickets maakt niet uit voor het netwerkverkeer en het geheugen van de applicatie. De database kan bovendien een index gebruiken.
Alternatief: alle tickets laden en in C# tellen. Dat werkt bij honderden tickets, maar het schaalt niet, want elk document gaat over het netwerk.

2. `$match` eerst, `$lookup` pas na `$group`
Wat: Bij een gewone medewerker staat `$match` op `reportedBy` vooraan. In de werkverdeling groepeert de pipeline eerst per behandelaar en doet daarna pas de `$lookup`.
Waarom: Het `$match` kan de index op `reportedBy` gebruiken en laat de rest van de pipeline een kleinere set verwerken. Door eerst te groeperen is er één lookup per medewerker en niet één per ticket.

Alternatief: `$lookup` vóór `$group`.

Dat is onnodig duur.

3. De categorie wordt afgeleid met een `$switch`
Wat: Status Open telt altijd als open. Status Resolved, of een gevulde `resolvedAt`, telt als resolved. Gesloten zonder `resolvedAt` is closed zonder oplossing. Al het andere, zoals In_Progress, is nog niet opgelost en telt als open. De status wordt vergeleken met `$toLower`.
Waarom: De opdracht vraagt om drie percentages, en het verschil tussen “gesloten en opgelost” en “gesloten zonder oplossing” zit niet in `status` maar in `resolvedAt`. Open wint altijd, omdat een heropend ticket zijn oude `resolvedAt` houdt. `$toLower` is nodig omdat de brondata andere hoofdletters gebruikte dan de applicatie. In_Progress telt als open op basis van de opdracht (“open, not yet solved”), afgestemd met Mart.
Alternatief: groeperen op `status`. Dan zitten gesloten tickets met en zonder oplossing in dezelfde groep.

4. Ontbrekende categorieën worden in C# aangevuld met 0
Wat: `$group` geeft geen rij voor een lege categorie. Een lus over de drie categorieën voegt die toe met 0 en sorteert in vaste volgorde.
Waarom: Het dashboard toont altijd drie cijfers. Bij nul tickets is er geen deling door nul (de pipeline geeft dan geen rijen) en geen kapotte weergave, maar een melding.

Alternatief: dit in de pipeline oplossen met `$facet` of `$setWindowFields`. Dat kan, maar het is lastiger te lezen en te controleren voor drie rijen.

5. Persoonlijk of globaal wordt bepaald via de claims

Wat: Het id van de ingelogde gebruiker komt uit de claims. Een gewone medewerker krijgt het persoonlijke dashboard, de service desk het globale dashboard met werkverdeling. Manager krijgt alle tickets te zien, medewerker kan alleen hun eigen tickets zien.

Waarom: Een id uit de URL kan iemand wijzigen om andermans tickets te zien. De rol bepaalt wat de server toont, en niet wat de browser vraagt.

Alternatief: een `userId` als URL-parameter. Dat gebruikte ik tijdens het ontwikkelen, maar het is onveilig.

6. Prioriteit wordt als getal opgeslagen

Wat: `Priority` staat in MongoDB als getal in de volgorde van de enum (Low=0 tot Urgent=3).

Waarom: Tekst sorteert alfabetisch (High, Low, Medium, Urgent), dus Low zou boven Medium staan. Een getal sorteert goed en past in een index. De waarden volgen de volgorde in de enum. Een test legt die getallen vast, zodat een wijziging in de enum niet stilletjes de data verschuift.

Alternatief: tekst met een extra veld `priorityRank` (dubbele data die uit sync kan raken), of de rang per query berekenen met `$addFields` en `$switch` (geen index, sorteert in het geheugen).

7. Sorteren met tiebreakers en een omgekeerde volgorde

Wat: `TicketPrioritySorter` sorteert op prioriteit, dan `createdAt`, dan `_id`. Laag naar hoog is de exacte omkering van hoog naar laag.

Waarom: Zonder tiebreakers is de volgorde binnen één prioriteit niet vast, en kan een ticket bij paginering dubbel of nooit voorkomen. Een index is in beide richtingen leesbaar als alle sleutels omdraaien, dus één index bedient beide richtingen.

Alternatief: nieuwste eerst in beide richtingen. Dat vraagt een tweede index of een sortering in het geheugen. Keyset-paginering is sneller bij grote datasets, maar `skip` volstaat voor deze omvang.

Mart: ontwerpkeuzes en onderbouwing

Ik heb één query object (`TicketListQuery`) in plaats van een constant groeiende lijst aan parameters gebruikt, omdat elke nieuwe filteroptie (eerst status, daarna “mijn tickets”, zoeken, paginering) een extra parameter in elke laag zou zijn geweest. Dat is erg foutgevoelig, want twee `int?`-parameters achter elkaar kun je makkelijk verwisselen zonder dat de compiler het merkt. Met een object met benoemde properties (`init`) is het wel leesbaar en een nieuwe optie toevoegen heeft alleen invloed op de plek waar hij echt gebruikt wordt. Het alternatief is losse parameters (zoals `GetTickets` eerst had). Dat werkt volgens mij voor twee filters, maar niet meer voor zes.

De filtering op status, toegewezen medewerker en melder staat op één plek. `TicketRepository`, `TicketPrioritySorter` en `TicketSearcher` gebruiken allemaal dezelfde methode. Dit, omdat eerst dezelfde filtercode drie keer in de code stond. Dit is redundant en er kunnen fouten gemaakt worden. Nu is de rolefilter altijd met AND gecombineerd met het zoekfilter dus zoeken kan nooit meer opleveren dan de gewone lijst. Eén plek betekent ook één plek om te testen en te wijzigen (DRY).

Ik heb een zoekfunctie gemaakt in een eigen klasse (TicketSearcher, met interface ITicketSearcher), los van de rest van de ticketcode. Je kunt zoeken op woorden in titel en omschrijving, hoofdletterongevoelig, en woorden combineren met AND en OR. “printer AND offline OR wifi” wordt gelezen als (printer en offline) of wifi. AND tussen twee woorden is de standaard, dus een zoekopdracht zonder operators werkt ook. De tekst wordt in ParseSearchText omgezet naar groepen. Die methode is public en static, zodat ik hem kan testen zonder database. De resultaten staan op nieuwste eerst met paginering (maximaal 100 per pagina), en de omschrijving wordt niet meegestuurd omdat de lijst die niet toont. `Regex.Escape` is voor veiligheid. Elk zoekwoord wordt eerst ge-escaped en daarna als hoofdletterongevoelige regex gebruikt op titel en omschrijving. Dit omdat, er zonder escaping de invoer van de gebruiker zelf een reguliere expressie is. Wie `(` of `.*` typt, krijgt een foutmelding of een heel dure query (regex-injectie / ReDoS). Met `Regex.Escape` wordt de tekst letterlijk genomen. Daarbovenop is de zoektekst beperkt tot 100 tekens en 20 woorden, zodat niemand de database kan belasten met een enorme zoekopdracht. Een alternatief is een `$text`-index in plaats van regex. Een tekstindex geeft stemming en relevantiescore en is sneller bij veel data. Hij vindt alleen geen deel van een woord (“print” in “printer”) en AND tussen woorden werkt alleen met een omweg met aanhalingstekens. Atlas Search is er niet op een lokale MongoDB. Regex kan geen index gebruiken en scant de hele collectie, maar dat is prima voor de omvang van dit project. Bij groei is een tekstindex de volgende stap.

Samengestelde indexen met `createdAt` als laatste veld (eerst gelijkheid, dan sortering). Bijvoorbeeld `reportedBy + createdAt` en `status + priority + createdAt + id`. De velden waarop met `=` gefilterd wordt staan voorop, het sorteerveld staat achteraan. Dit, omdat MongoDB een index alleen goed kan gebruiken voor filteren en sorteren tegelijk als de volgorde van de velden past bij de query. Eerst de gelijkheid velden (status, melder), daarna het sorteerveld. Dan komen de documenten uit de index al in de juiste volgorde en hoeft de database niet alles in het geheugen te sorteren. `Id` staat bij de prioriteitssortering als laatste, zodat de volgorde vast ligt wanneer twee tickets dezelfde prioriteit en tijd hebben (anders kunnen tickets bij paginering dubbel of helemaal niet voorkomen). De prioriteitsindexen stonden eerst in `EnsureIndexesAsync` van de sorter, maar die methode werd nergens aangeroepen, dus de indexen bestonden niet. Ze staan nu in `MongoIndexInitializer`, dat wel bij het opstarten wordt aangeroepen.

Unieke `ticketId`-index met retry. `ux_ticketId` is een unieke index. `CreateTicket` leest het laatste ticketnummer, telt er één bij op en probeert op te slaan. Bij een `DuplicateKey`-fout probeert hij het tot drie keer opnieuw met een nieuw nummer. Dit, omdat “Lees het hoogste nummer en tel er 1 bij op” niet veilig is als twee medewerkers tegelijk een ticket aanmaken. Ze kunnen dan hetzelfde nummer krijgen. De unieke index laat

de database dat afdwingen, ook als de applicatiecode het niet opvangt. De retry zorgt dat de tweede gebruiker geen foutpagina krijgt, maar gewoon het volgende nummer. Na drie mislukte pogingen volgt een duidelijke fout in plaats van een endless loop. Een alternatief zou een tellercollectie in plaats van “laatste id + 1” kunnen zijn geweest. Een aparte collectie met een teller die je ophoogt (`findOneAndUpdate` met `$inc`) geeft nooit dubbele nummers en heeft geen retry nodig. Ik heb er niet voor gekozen, omdat het een extra collectie en extra code betekent. De unieke index met retry lost hetzelfde probleem op met minder code. Bij veel gelijktijdige gebruikers zou de teller wel een betere keuze zijn.

Emre: ontwerpkeuzes en onderbouwing

Mijn deel is het ticket formulier en alles eromheen: aanmaken (ook namens iemand), bewerken met status, verwijderen en Escalate/Close. Dit zijn mijn keuzes.

1. Validatie als data annotations op het model

De regels staan als `[Required]` en `[String Length]` op Title (3 tot 100 tekens) en Description (10 tot 2000 tekens), met duidelijke meldingen. De formulieren tonen die bij het veld. Omdat Create en Edit hetzelfde model gebruiken, staan de regels maar op één plek. De server blijft de echte controle, want alles in de browser is te omzeilen.

Alternatief: validatie alleen in de browser (niet veilig) of aparte viewmodels per formulier. Dat is netter, maar je krijgt een stapel klassen met bijna dezelfde velden plus omzetcode. Dat vond ik voor dit project te veel.

2. Bind-lijsten tegen over-posting

Elke POST-actie noemt zelf welke velden gebonden mogen worden. Bij Create zijn dat `Title`, `Description`, `Type`, `Priority`, `Deadline` en `ReportById`. Bij Edit komen `Status` en `AssignedToId` erbij. Zonder lijst kan een medewerker bij het aanmaken zelf `Status=Closed` meesturen en dan wordt dat gewoon opgeslagen. Bij een gewone medewerker overschrijf ik `ReportById` met het ID uit de claims. Bij Service Desk controleer ik of het een actieve medewerker is.

Alternatief: viewmodels met alleen de toegestane velden. Dat is stevig, maar meer werk. Bij een Bind-lijst moet je hem wel zelf bijhouden, al is een nieuw veld standaard veilig.

3. Deadline op de server gecontroleerd

De controller checkt bij Create dat de deadline is ingevuld en niet voor vandaag ligt. Het datumveld heeft in de browser ook een `min`, maar dat is alleen een hulpmiddel. De regel staat bewust niet op het model, want dan zou je een ticket met een verlopen deadline niet meer kunnen bewerken.

Alternatief: een eigen validatie attribuut (netter, maar het geldt dan ook in Edit) of alleen `min` in de HTML (te omzeilen).

4. Verwijderen pas na een bevestigingspagina

De Delete-pagina toont het ticket en vraagt of je het zeker weet. Pas de POST (Delete Confirmed) verwijdert echt, met anti-forgery-token en alleen voor Service Desk. Een GET mag niets wijzigen, anders kan een link of een voorlezende browser tickets weggooien. Een harde verwijdering kan hier, want tickets verwijzen naar medewerkers en niet andersom.

Alternatief: een `confirm()` op de knop. Dat gebruik ik wel bij Escalate en Close, maar voor iets onomkeerbare vond ik een aparte pagina beter.

5. Compare-and-set bij UpdateStatus

`UpdateStatus(id, expectedCurrent, newStatus)` past alleen toe als het ticket nog de status heeft die we net lazen, en doet dat in één `UpdateOne`. Zo kunnen twee Servicedesk-medewerkers elkaar niet overschrijven, bijvoorbeeld door een ticket te escaleren dat een collega net heeft gesloten. Dan verandert er niets en krijgt de gebruiker een melding.

Alternatief: alleen op ID filteren (de laatste wint) of een versienummer/transactie. Dat is robuuster, maar voor één statusveld te zwaar.

Beperking: het bewerkscherm heeft deze controle niet, daar wint nog de laatste.

16. Gebruikersflow en vormgeving

De gedeelde layout en `site.css` geven alle pagina's dezelfde kaarten, tabellen, knoppen en formulieren. Het dashboard heeft een eigen `dashboard.css` (gestapelde balk plus percentages, met een lege-toestand-melding). Bootstrap is gekozen voor responsiviteit en snelle ontwikkeling. De lay-out is door Emre. Anderen passen gedeelde bestanden alleen na overleg aan.